

Backend Process Flow Documentation for Flowchart Making

This document explains how data moves through the system. Use this to create flowcharts for each feature.

1. Authentication Flow (Login & Registration)

Registration Process

When a new user signs up, the system receives their name, email, and password. It first checks if the email is already in use. If the email is available, the system encrypts the password so it cannot be read if hacked. It then assigns the "employee" role by default and saves the user as "active" in the database. Finally, it confirms to the user that registration was successful. If the email already exists, the system rejects the registration with an error message.

Login Process

When a user attempts to log in, the system receives their email and password. It looks up the email in the database. If found, it compares the provided password with the stored encrypted version. If both match, the system checks if the account is active (not deactivated). For active accounts, the system generates a time-limited token (expires after 1 hour) and sends it back to the user along with their basic information. This token must be included in all future requests to prove who they are. If any check fails (wrong email, wrong password, or deactivated account), the system returns an error.

2. Role-Based Access Control

Every request that requires authentication includes the user's token. The system first verifies the token is valid and not expired. It then extracts the user's role (admin or employee) from the token.

For admin-only actions, the system checks if the role is "admin". If yes, the request proceeds. If no, the system immediately rejects the request with a "forbidden" error. Regular employees cannot access admin functions even if they know the URL.

For employee-accessible actions, the system allows the request to proceed as long as the token is valid, regardless of role (except for explicit restrictions).

3. User Management (Admin Only)

Viewing All Users

When an admin requests the user list, the system queries the database for all users. It joins the roles table to get each user's role name (admin or employee). The system returns each user's ID, name, email, role, status (active/inactive), and creation date. This powers the admin's user management table.

Viewing a Single User

The system receives a user ID from the admin, queries the database for that specific user, and returns their details.

Creating a New User

The admin provides a name, email, and role ID. The system first checks that the email is not already in use. It then automatically generates a secure random password. The password is encrypted before storage. The system saves the user with "active" status. The response includes the plain-text temporary password so the admin can give it to the new user.

Updating a User

The admin provides a user ID and any combination of new name, new email, new role, or new status. The system updates only the fields provided. The password cannot be changed through this endpoint (separate reset endpoint exists). The system confirms the update was successful.

Activating/Deactivating a User

The admin provides a user ID and a status (active or inactive). The system updates that user's status. Deactivated users cannot log in at all. Their existing tokens become useless. Their documents remain in the system but they cannot access anything.

Resetting a User's Password

The admin provides a user ID. The system generates a new secure random password, encrypts it, and updates the user's password in the database. The response includes the plain-text temporary password. The user's old password immediately stops working.

4. User Self-Service

Viewing Own Profile

The system extracts the user's ID from their token (no need to provide it manually). It queries the database for that user's information and returns their name, email, role, status, and creation date. The user cannot see other users' profiles.

Updating Own Profile

The system receives the user's ID from the token. The user provides a new name and/or new email. The system updates only those fields. Users cannot change their own role or status through this endpoint. If changing email, the system checks the new email is not already used by another user.

Changing Own Password

The user provides their current password and a new password. The system verifies the current password matches the stored encrypted version. If correct, it encrypts the new password and

saves it. The user must then use the new password for future logins. The system requires the new password to be at least 6 characters long.

5. Folder Management

Viewing Folders

Any authenticated user can request all folders. The system queries the database and returns each folder's ID, name, permissions (public/private/restricted), who created it, and when. The system also returns folders created by a specific user when requested.

Creating a Folder

The user provides a folder name and optional permission setting (defaults to public). The system checks that no folder with the same name already exists. It saves the folder with the current user's ID as the creator. The response confirms creation with the new folder's ID.

Updating a Folder

The user provides a folder ID and either a new name or new permissions. The system updates the folder. If changing name, it checks the new name is not already taken. Users can update any folder regardless of who created it (no ownership restrictions).

Deleting a Folder (Admin Only)

The admin provides a folder ID. The system locates the folder and checks how many documents are inside. It then deletes the folder from the database. Due to database settings, all documents inside that folder are automatically deleted as well. The system also deletes the physical files from the uploads folder. The response tells the admin how many documents were affected.

6. Document Management

Viewing Documents

When a user requests all documents, the system must filter based on access rights. For admins, it returns every document. For employees, it returns only documents they can access based on folder permissions or explicit grants. The system joins folder and user tables to provide complete information including folder name and uploader name.

Accessible Documents Endpoint

This is the recommended endpoint for the main document list. The system automatically filters documents based on the logged-in user's access rights without requiring any additional parameters.

Viewing Documents by Folder

The user provides a folder ID. The system returns all documents inside that specific folder, still applying access filtering.

Searching Documents

The user provides a search keyword. The system searches the title, description, and category fields for matches. It uses partial matching (contains the keyword, not exact match). Results are filtered by access rights before being returned.

Uploading a Document

This is a multi-step process. The system receives a file plus metadata (title, description, category, folder ID, optional accessibility override). It first verifies the folder exists. It then checks no document with the same title exists in that folder. The file is validated (type must be PDF, DOC, DOCX, TXT, XLS, or XLSX; size must be under 10MB). The system generates a unique filename to prevent conflicts and saves the file to the uploads folder. It then stores the document record in the database with the file path, size, type, and uploader ID. The system also logs this upload action. If any validation fails, any partial file saved is deleted.

Updating Document Metadata

The user provides a document ID and any combination of new title, description, category, or accessibility setting. The system updates only the fields provided. The actual file content cannot be changed through this endpoint (would require re-uploading).

Deleting a Document (Admin Only)

The admin provides a document ID. The system retrieves the file path from the database. It deletes the document record from the database. It then deletes the physical file from the uploads folder. The system logs this deletion. Both database and file system must be successfully cleaned up.

Downloading a Document

This is the most complex flow. The system receives a document ID and the user's token. It retrieves the document and its folder information. It first checks if the physical file still exists on disk. Then it runs the permission check (see section 8). If access is granted, it logs the download action. It sets response headers to tell the browser to download (not display) the file. Finally, it streams the file from disk to the user in chunks. If access is denied, it returns a forbidden error.

7. Document Permissions

Permission Hierarchy (Checked in Order)

When checking if a user can access a document, the system evaluates conditions in this specific order:

First, if the user has the admin role, access is granted immediately without further checks.

Second, if the folder containing the document has "public" permission, any authenticated user gets access.

Third, if the user is the person who uploaded the document, they get access.

Fourth, the system checks the document permissions table for an explicit entry granting this user access to this document. If found, access is granted.

Finally, if none of these conditions are true, access is denied.

Checking Access

Users can send a document ID to a dedicated endpoint. The system runs the same permission hierarchy and returns true or false, along with the reason for the result (admin, public folder, uploader, explicit permission, or no permission).

Granting Access (Admin Only)

The admin provides a document ID and a user ID. The system verifies both exist. It checks that the user doesn't already have access. It then creates a record in the document permissions table linking that user to that document with a permission type (view or download). The system logs this action.

Revoking Access (Admin Only)

The admin provides a document ID and a user ID. The system finds and deletes the permission record. The user immediately loses access to that document. The system logs this action.

Viewing Access Lists (Admin Only)

The admin can request all users who have explicit access to a specific document. The system returns a list of user names, emails, and their permission types. The admin can also request all documents a specific user has access to.

8. Audit Logs

What Gets Logged

Every significant action automatically creates a log entry. This includes: login, logout, upload, download, document edit, document delete, and permission changes. Each log records which user performed the action, what action was performed, which document was involved (if applicable), and the exact timestamp.

How Logging Works

When a user performs an action, the system immediately inserts a record into the logs table. This happens automatically within the same database transaction as the action itself. If the action fails, the log is not created. Logs cannot be deleted or modified by any user, including admins (maintains audit integrity).

Viewing Logs (Admin Only)

Admins can request logs with various filters. The system returns paginated results (20 per page by default). Available filters include: date range (start and end dates), action type (login, logout, upload, download, edit, delete), and page number. The system also provides formatted time-ago strings for display.

Log Statistics (Admin Only)

The system analyzes all logs and returns summary data. This includes counts of each action type, daily activity for the last 7 days, the 5 most active users, and the 5 most accessed documents. This powers the dashboard charts.

9. Dashboard Analytics (Admin Only)

Main Statistics

The system runs several database counts in parallel. It counts total users, active users, inactive users, total documents, total folders, total downloads, total uploads, and total logins. It also calculates total storage used by summing all file sizes. The system formats the storage size into human-readable form (KB, MB, GB) and calculates the percentage of active users. All these numbers are returned in a single response for the dashboard.

Chart Data

The system receives a parameter for number of days (7 or 30). It calculates the date range for the requested period. It queries the database for daily counts of uploads, downloads, logins, and new user registrations over that period. It returns arrays where each entry has a date and a count. Days with zero activity are included as zero values. This allows the frontend to draw line or bar charts without additional processing.

Recent Activity Feed

The system retrieves the most recent logs (default 20, adjustable). For each log, it generates a human-readable message like "John uploaded 'Report.pdf'". It also calculates how long ago each action happened (e.g., "2 hours ago", "3 days ago"). This creates an activity timeline similar to social media feeds.

Top Users and Documents

The system queries for the most active users (based on total log entries) and the most downloaded documents (based on download log counts). Both queries default to top 5 but accept a limit parameter.

10. File Storage

Where Files Live

Uploaded files are stored in a folder called "uploads" inside the backend project. The database stores only the file path, not the file itself. This keeps the database small and fast.

File Naming

When a file is uploaded, the system generates a unique name combining the current timestamp, a random number, and the original filename. This prevents two files with the same name from overwriting each other.

File Validation

Before saving any file, the system checks the file type against an allowed list. It also checks the file size is under 10MB. Rejected files are immediately discarded, not saved.

File Retrieval

When a user downloads a file, the system reads the file from disk and streams it to the user in small chunks. This is memory-efficient even for large files. The file is never loaded entirely into memory.

File Deletion

When a document is deleted from the database, the system also finds and deletes the physical file from disk. This prevents orphaned files from accumulating.